

Mise en ligne CI/CD chez la cliente — architecture DMZ Docker

Suivi pas-à-pas. Coche chaque case au fur et à mesure. Cible : VPS Contabo cliente (Traefik v3.3 + Postgres/MinIO existants).

0. L'approche en un coup d'œil

Deux environnements, une seule base de code, pilotés par la **branche Git** :

Environnement	Branche	Serveur	Reverse-proxy	Domaines
Recette (nous)	staging	Contabo (nous)	nginx mutualisé	*.gathe-finance.horus-lab.com
Production (cliente)	main	Contabo cliente — DMZ Docker	Traefik v3.3 existant	*.gathe-finance.com

Flux : dev → merge staging (recette auto) → on teste → **PR staging** → **main** → au merge : CI build les images :main (domaines cliente) → deploy.yml se connecte en SSH → docker compose pull + up → le Traefik existant route les sous-domaines → smoke tests → **rollback auto** si échec.

Principe clé — on se BRANCHE sur l'existant (on ne duplique rien)

Le serveur cliente est une **DMZ durcie** : Traefik v3.3 (seul exposé sur 80/443), et un cœur de données isolé (Postgres 16 + MinIO) sur un réseau `data` coupé d'Internet. Notre application devient une **3^e stack** qui se rattache à ces réseaux **sans relancer** ni Traefik, ni Postgres, ni MinIO.

Notre conteneur	Réseaux	Se connecte à
backend (Django)	edge + data + internal	leur Postgres + MinIO ; exposé Traefik (api . cms .)
qcluster (worker)	data + internal	leur Postgres
site / portail / admin	edge + internal	le backend ; exposés Traefik (app . portail . admin .)

On ne lance PAS : notre Traefik, notre Postgres (db), backup , beszel — tout existe déjà. Redis n'est pas utilisé par l'app.

Mapping des sous-domaines

Sous-domaine	Service	Géré par nous ?
app . portail . admin . api . cms .	Nos 5 services Gathé	✅ oui
traefik . / console . / s3 .	Infra DMZ existante (Traefik, MinIO console + S3)	❌ existant — ne pas toucher

1. Pré-requis

- ☐ **Accès SSH** au VPS cliente (IP, user, port).
- ☐ **Coordination Broad Range** (admins DMZ) : création base + accès réseau `data` + clés MinIO.
- ☐ **Contrôle DNS** de `gathe-finance.com` (enregistrements A).
- ☐ **PAT GitHub** scope `read:packages` (images privées GHCR).
- ☐ **Secrets applicatifs** : `DJANGO_SECRET_KEY` , clé API Brevo, (Tara si paiement réel).
- ☐ **Clé SSH de déploiement** dédiée (ed25519).

2. Côté GitHub — configuration

Emplacement: `Repo → Settings → Secrets and variables → Actions` .

2.1 — Variables de dépôt (onglet « Variables »)

Les URLs du front sont figées au build des images `:main` . Ces variables doivent être au niveau Repository.

Variable	Valeur
<code>CLIENT_SITE_DOMAIN</code>	<code>app.gathe-finance.com</code>
<code>CLIENT_PORTAL_DOMAIN</code>	<code>portail.gathe-finance.com</code>
<code>CLIENT_API_DOMAIN</code>	<code>api.gathe-finance.com</code>
<code>CLIENT_ADMIN_DOMAIN</code>	<code>admin.gathe-finance.com</code>
<code>CLIENT_CMS_DOMAIN</code>	<code>cms.gathe-finance.com</code>

- ☐ Les 5 variables sont créées.

2.2 — Environment « production » → Secrets

Isole les secrets SSH de la cliente (+ règle de protection possible avant chaque prod).

Secret	Valeur
<code>CLIENT_VPS_HOST</code>	IP / hostname du VPS cliente
<code>CLIENT_VPS_USER</code>	utilisateur SSH
<code>CLIENT_VPS_SSH_PRIVATE_KEY</code>	clé privée ed25519 dédiée
<code>CLIENT_VPS_DEPLOY_PATH</code>	(opt.) <code>/srv/gathe</code>

- ☐ Secrets créés dans l'Environment `production` .

Ne pas toucher aux secrets `VPS_*` existants : ils servent la recette (Contabo, branche `staging`).

3. Côté serveur cliente — bootstrap (une fois)

En SSH sur le VPS cliente. Étapes 3.1–3.3 souvent réalisées avec **Broad Range**.

3.1 — Relever les réseaux + le Postgres/MinIO + le certresolver

Nos conteneurs se rattachent aux réseaux `edge` (Traefik) et `data` (bases). Il faut leurs **noms Docker exacts**, l'alias joignable du Postgres, et le nom du certresolver de leur Traefik.

```
docker network ls                # noms exacts de edge et data
docker inspect postgres --format '{{.Name}} {{json .NetworkSettings.Networks}}'
docker inspect minio --format '{{.Name}} {{json .NetworkSettings.Networks}}'
docker inspect traefik | grep -i certificatesresolvers | head
# ou : grep -ri certresolver /srv/edge
```

☐ `EDGE_NETWORK = _____` · `DATA_NETWORK = _____` · `PG_HOST(alias) = _____` ·
`TRAEFIK_CERTRESOLVER = _____`

3.2 — Créer la base `gathe_prod` + le user `gathe` (leur Postgres)

On réutilise leur Postgres, avec une base + un utilisateur dédiés (isolation logique).

```
docker exec -it postgres psql -U postgres -c "CREATE USER gathe WITH PASSWORD '<MDP_FORT>';"
docker exec -it postgres psql -U postgres -c "CREATE DATABASE gathe_prod OWNER gathe;"
```

☐ Base + user créés → `DATABASE_URL=postgres://gathe:<MDP>@<PG_HOST>:5432/gathe_prod`

3.3 — Créer le bucket + les clés MinIO (médias)

Les fichiers médias (justificatifs, photos) vont dans leur MinIO via l'API S3.

```
# via la console MinIO (console.gathe-finance.com) ou le client `mc` :
mc mb local/gathe-media
mc admin user svcacct add local <admin> # génère access key + secret key
```

☐ Bucket `gathe-media` + `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` notés, + `AWS_S3_ENDPOINT_URL` (endpoint interne MinIO sur `data`, ou `https://s3.gathe-finance.com`).

3.4 — Cloner le dépôt

```
sudo mkdir -p /srv/gathe && cd /srv/gathe
git clone https://github.com/EdwinTchakounte/GatheFinance_project.git .
git checkout main
```

☐ Repo cloné dans `/srv/gathe` sur `main`.

3.5 — Remplir `infra/.env.prod` (secrets — jamais commité)

```
cp infra/.env.prod.client.example infra/.env.prod
nano infra/.env.prod
```

Renseigner : `__CLIENT_DOMAIN__` → `gathe-finance.com` , `EDGE_NETWORK` , `DATA_NETWORK` ,
`TRAEFIK_CERTRESOLVER` , `DATABASE_URL` (étape 3.2), `AWS_S3_*` (étape 3.3), `DJANGO_SECRET_KEY` ,
`BREVO_API_KEY` , `GATHE_IMAGE_TAG=main` .

☐ Aucun `__CHANGE_ME__` restant.

3.6 — Connexion au registre GHCR

```
echo <PAT_read_packages> | docker login ghcr.io -u <user_github> --password-stdin
```

☐ `Login Succeeded` .

3.7 — DNS : enregistrements A

Traefik émet les certificats TLS par challenge → les domaines doivent pointer sur le serveur AVANT le 1er démarrage.

Type	Nom	Valeur
A	<code>app / portail / admin / api / cms</code>	IP du VPS cliente

☐ 5 A records créés (`dig app.gathe-finance.com` renvoie l'IP).

3.8 — Autoriser la clé de déploiement

```
ssh-copy-id -i ./gathe_deploy_client.pub <CLIENT_VPS_USER>@<IP>
# puis coller la clé PRIVÉE dans le secret GitHub CLIENT_VPS_SSH_PRIVATE_KEY
```

☐ Clé publique dans `~/ .ssh/authorized_keys` , clé privée dans le secret GitHub.

4. Premier déploiement + seeds

4.1 — Démarrer (1re fois en manuel pour valider)

On démarre **uniquement** nos 5 services applicatifs (`--no-deps`), rattachés aux réseaux existants.

```
cd /srv/gathe/infra
DC="docker compose -f docker-compose.prod.yml -f docker-compose.client.yml --env-file .env.prod"
$DC pull backend qcluster site portal admin
$DC up -d --no-deps backend qcluster site portal admin
```

☐ Conteneurs démarrés ; Traefik émet les certificats (patiente 1–2 min).

4.2 — Migrations + seeds (idempotents)

```
$DC exec backend python manage.py migrate
$DC exec backend python manage.py bootstrap_site
$DC exec backend python manage.py seed_blog
$DC exec backend python manage.py seed_email_templates --force
$DC exec backend python manage.py seed_fees
$DC exec backend python manage.py seed_rates
$DC exec backend python manage.py seed_app_settings
$DC exec backend python manage.py seed_q_schedules
```

☐ Migrations + seeds passés sans erreur.

4.3 — Super-administrateur

```
$DC exec backend python manage.py createsuperuser
```

☐ Superuser créé.

Ensuite, tout est automatique : chaque merge dans main rejoue pull + up -d --no-deps via deploy.yml .
Plus de manip serveur au quotidien.

5. Vérification (recette de mise en ligne)

Contrôle	Attendu	OK
<code>curl -I https://api.gathe-finance.com/healthz/</code>	200 + cert valide	☐
<code>app.gathe-finance.com</code>	vitrine	☐
<code>portail.gathe-finance.com</code>	connexion membre	☐
<code>admin.gathe-finance.com</code>	connexion back-office	☐
<code>cms.gathe-finance.com/admin/</code>	Wagtail (302 → login)	☐
Upload d'un média → visible	fichier servi depuis MinIO	☐
Webhook Tara	<code>PUBLIC_BASE_URL=https://api.gathe-finance.com</code>	☐

6. Le cycle courant (après le bootstrap)

1. Développer → merger dans staging → **recette auto (horus-lab)**
2. Tester la recette.
3. **PR staging → main** → merger → **prod cliente auto**
4. Healthcheck backend KO → **rollback automatique** sur le tag précédent.

7. Dépannage rapide

Symptôme	Cause probable → action
404 Traefik sur un sous-domaine	mauvais <code>EDGE_NETWORK</code> / label absent → vérifier <code>docker-compose.client.yml</code> + <code>traefik.docker.network</code> (étape 3.1)
Backend « could not connect to server »	<code>DATABASE_URL</code> / <code>DATA_NETWORK</code> / alias Postgres faux → étapes 3.1–3.2. Vérifier que le backend est bien sur le réseau <code>data</code> .
Cert TLS invalide	DNS pas propagé, ou mauvais <code>TRAEFIK_CERTRESOLVER</code> → étapes 3.1 + 3.7
Médias non enregistrés / 403 S3	bucket/clé MinIO KO ou <code>AWS_S3_ENDPOINT_URL</code> injoignable → étape 3.3
<code>pull access denied</code> GHCR	PAT manquant/expiré → refaire <code>docker login ghcr.io</code> (étape 3.6)
Déploiement GitHub « ignoré »	<code>CLIENT_VPS_HOST</code> non renseigné → étape 2.2
Front affiche les mauvaises URLs	images <code>:main</code> buildées avant les variables <code>CLIENT_*</code> → re-lancer la CI
<code>db</code> ou <code>traefik</code> démarre par erreur	oubli du <code>--no-deps</code> ou des noms de services au <code>up</code> → n'utiliser que la commande de l'étape 4.1

Gathé Finance — Runbook déploiement production (DMZ Docker cliente). Document de suivi interne. Réf. dépôt : `infra/DEPLOY_ENVIRONMENTS.md`, `infra/docker-compose.client.yml`, `infra/.env.prod.client.example`, `.github/workflows/deploy.yml`.